

OC (OpenShift Command Line Execution)

Milenko Burlica

28 July, 2026

Table of Contents

1	💡 Das Entkopplungs-Prinzip (Separation of Concerns).....	4
2	Globales Feature: condition (Bedingte Ausführung)	5
3	YAML	6
3.1	Engine-spezifische Parameter	6
3.2	Unterstützte Operationen (op)	6
3.2.1	1. Operation: EXEC (Kommando im Pod ausführen)	6
4	YAML	7
4.1	2. Operationen: PUT und GET (Datei-Transfer)	7
5	YAML	8
5.1	3. Operation: ASSERT (Validierung).....	8
6	YAML	9
6.1	4. Operation: BUFFER (Datenextraktion)	9
7	YAML	10
7.1	🔧 Konsole-Logging & Live-Ausgabe (Log Alignment).....	10
8	Plaintext.....	11
8.1	💡 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling	11
8.1.1	1. Schutz von Dynamic-Tags { }	11
8.1.2	2. Verschachtelte Anführungszeichen bei command.....	11

Die OC-Engine dient zur Ausführung von OpenShift CLI-Befehlen direkt innerhalb von Containern (Pods) sowie für den Datentransfer zwischen dem lokalen Windows-System und dem OpenShift-Cluster. Sie reagiert dediziert auf den Testschritt-Typ `type: OC`.

1 Das Entkopplungs-Prinzip (Separation of Concerns)

Die Engine strikt trennt die **Ausführung** von der **Validierung**:

1. Ein Befehl wird **einmalig ausgeführt** (`op: EXEC` , `PUT` oder `GET`) und das Ergebnis unter einem eindeutigen Namen (`response`) im Buffer zwischengespeichert.
2. Anschließend können **beliebig viele Prüfungen** (`op: ASSERT`) oder **Datenextraktionen** (`op: BUFFER`) auf diesem gespeicherten Ergebnis durchgeführt werden, ohne den Pod erneut zu belasten.

2 Globales Feature: condition (Bedingte Ausführung)

Mit dem optionalen Feld `condition` kann gesteuert werden, ob ein OC-Testschritt ausgeführt werden soll. Wenn die Bedingung nicht zutrifft (`false`), wird der Schritt übersprungen (**SKIPPED**) und der Test läuft fehlerfrei weiter.

3 YAML

```
# Führt den Befehl nur aus, wenn die TestCase-Variable "run_openshift_tests" auf
true steht
- type: OC
  condition: '{{B[run_openshift_tests]}}' == 'true'
  op: EXEC
  pod: "my-backend-service"
  command: "ls -la /app/logs"
  response: logs_list
```

3.1 Engine-spezifische Parameter

Neben den Standardfeldern besitzt die OC-Engine spezielle Steuerungs-Parameter im YAML:

- **pod (String, Pflichtfeld):** Der Name des Ziel-Pods oder Services im Cluster. Die Engine sucht im Cluster automatisch nach dem aktuell laufenden Pod (`status.phase=Running`), der mit diesem Präfix beginnt (automatische Auflösung der dynamischen Kubernetes-Suffixe).
- **namespace (String, Optional):** Der OpenShift-Namespace (Projekt). Wird dieses Feld weggelassen, zieht das Framework automatisch den in der globalen Konfigurationsdatei definierten Standard-Namespace (`env.openshift.namespace`).
- **from / to (String):** Pfadangaben, die exklusiv für den Datei-Up- und Download (`op: PUT` / `op: GET`) benötigt werden.

3.2 Unterstützte Operationen (op)

3.2.1 1. Operation: EXEC (Kommando im Pod ausführen)

Führt ein Linux-Kommando isoliert im Ziel-Pod über eine `bash -c` Shell aus.

- **Automatisches Lazy-Login:** Sollte die OpenShift-Session abgelaufen sein oder ein *Unauthorized/Forbidden*-Fehler auftreten, erkennt die Engine dies automatisch. Sie führt im Hintergrund selbstständig ein `oc login` mit den verschlüsselten System-Credentials aus, setzt das Projekt zurück und wiederholt den Befehl transparent für den Benutzer.

4 YAML

```
- type: OC
  op: EXEC
  namespace: "${E[env.openShift.namespace]}" # Dynamisch aus Umgebung oder statisch
  überschrieben
  pod: "auth-service"
  command: "java -version"
  response: java_out
```

4.1 2. Operationen: PUT und GET (Datei-Transfer)

Diese Operationen spiegeln den nativen `oc cp`-Befehl für den Datentransfer wider.

- `op: PUT` : Kopiert eine lokale Datei vom Quellpfad (`from`) in das Dateisystem des Pods (`to`).
- `op: GET` : Lädt eine Datei aus dem Pod (`from`) herunter auf das lokale System (`to`).
- **Intelligente Pfadbereinigung**: Unter Windows extrahierte Laufwerksbuchstaben (wie `C:`) werden von der Engine für die OpenShift-Syntax automatisch bereinigt (`\temp\...`), um Transferfehler zu vermeiden.

5 YAML

```
# Datei in den Pod hochladen (PUT)
- type: OC
  op: PUT
  pod: "batch-processor"
  from: "C:\TestData\payload.json"
  to: "/app/input/payload.json"

# Datei aus dem Pod herunterladen (GET)
- type: OC
  op: GET
  pod: "batch-processor"
  from: "/app/output/result.csv"
  to: "C:\TestResults\result.csv"
  response: transfer_status
```

5.1 3. Operation: ASSERT (Validierung)

Prüft den Inhalt oder den Status (Exit-Code) einer zuvor gespeicherten OC-Response. Schlägt eine Prüfung fehl, bricht der gesamte Test sofort ab (**Fail-Fast**).

Aktion (action)	Beschreibung	Verwendung im YAML
EXITCODE	Prüft den Rückgabewert des Prozesses (Erfolg im Pod ist 0).	expected: 0
CONTAINS	Prüft, ob ein Text im Pod-Output existiert.	value: "Suchtext"
NOT_CONTAINS	Prüft, ob ein Text nicht im Pod-Output existiert.	value: "ERROR"
COUNT	Zählt, wie oft ein Text im Pod-Output vorkommt.	value: "Text" expected: 2

6 YAML

```
# Inhaltsprüfung auf dem zuvor gespeicherten "logs_list" Output
- type: OC
  op: ASSERT
  response: logs_list
  action: NOT_CONTAINS
  value: "Permission denied"
```

6.1 4. Operation: BUFFER (Datenextraktion)

Liest gezielt Daten aus einer gespeicherten OC-Response und legt sie als neue Variable (`name`) im TestCase ab, damit nachfolgende Schritte (auch für SQL, REST oder CMD) darauf zugreifen können.

- `TEXT` : Speichert den kompletten, getrimmten Text-Output.
- `FILTER` : Speichert nur Zeilen, die den Suchbegriff (`value`) enthalten.
- `LINE` : Extrahiert eine exakte Zeile über einen Index (beginnend bei `0` für die erste Zeile).
- `LAST_LINE` : Extrahiert eine Zeile relativ vom Ende (`value: 0` ist die letzte Zeile).

7 YAML

```
# Holt die letzte Zeile des Logs aus dem Buffer "logs_list"
- type: OC
  op: BUFFER
  response: logs_list
  action: LAST_LINE
  value: 0
  name: last_log_entry
```

7.1 Konsole-Logging & Live-Ausgabe (Log Alignment)

Die OC-Engine verfügt über ein vertikal ausgerichtetes Logging-System. Alle System-Tags (>>> OC EXEC , >>> ASSERT , >>> BUFFER , <<< OUT , >>> RESULT)

8 Plaintext

```
>>> OC EXEC      : [kvw-int-app // kvw-pst-pst-6bc7f8646b-nrvf9]: java -version
<<< OUT          : openjdk version "21.0.11" 2026-04-21 LTS
<<< OUT          : OpenJDK Runtime Environment (Red_Hat-21.0.11.0.10-1) (build
21.0.11+10-LTS)
>>> RESULT       : SUCCESS (Exit: 0)

>>> ASSERT       : [java_out] EXITCODE "0"
>>> RESULT       : SUCCESS (Exit: 0)

>>> BUFFER       : [java_out] Store entire output to variable [extracted_version]
<<< OUT          : openjdk version "21.0.11" 2026-04-21 LTS
>>> RESULT       : SUCCESS (Buffered)
```

8.1 💡 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling

Da OC-Befehle oft komplexe Linux-Argumente, Anführungszeichen oder JSON-Strings enthalten, müssen beim Schreiben der YAML-Dateien folgende Regeln beachtet werden:

8.1.1 1. Schutz von Dynamic-Tags { }

Wenn ein Feld mit einem Platzhalter beginnt, muss der gesamte Wert in Anführungszeichen gesetzt werden, da die geschweiften Klammern sonst den YAML-Parser blockieren.

- ❌ **Falsch:** `pod: {B[target_pod]}`
- ✅ **Richtig:** `pod: "{B[target_pod]}"`

8.1.2 2. Verschachtelte Anführungszeichen bei `command`

Da die Engine den Befehl intern in ein `bash -c "..."` Konstrukt einbettet, sind doppelte Anführungszeichen innerhalb des eigentlichen `command`-Feldes oft fehleranfällig.

- **Die beste Lösung – Block-Notation (|):** Nutze das Pipe-Zeichen für den Befehl. Dadurch kannst du im Textblock völlig frei mit einfachen (') und doppelten (") Anführungszeichen arbeiten, ohne etwas maskieren zu müssen.

YAML

```
- type: OC
  op: EXEC
  pod: "config-service"
  command: |
    grep "database" /app/config.properties | awk -F=' ' '{print $2}'
  response: db_config
```

- **Alternative – Einfache äußere Anführungszeichen:** Setze das gesamte `command`-Feld in einfache Anführungszeichen, falls es sich um einen kurzen Einzeiler handelt:

YAML

```
command: 'echo "Hello Cluster" > /tmp/test.txt'
```